

# Per-track dense retrieval route — routing the embedding recall to browsers

Branch `dense_rerank` (`chain: main <- dual_tracking <- stress_harness <- dense_rerank`).  
`RetrievalConfig.use_dense, src/retrieval.py, src/embed.py, AgentConfig.browsing_retrieval`.

## The hypothesis

`docs/team/stress_harness.md` and `docs/team/dense_rerank.md` between them locate the retrieval gap precisely: under paraphrase:heavy+browse-gated the **browsing** scenario loses the target from the candidate pool entirely in ~10 of 80 sessions (`never_retrieved`), while buying stays at ~0. Browsers open vague and stay vague, so the lexical routes have few verbatim tokens to match; buyers recite constraints and BM25 recalls them fine.

`docs/team/dense_rerank.md` showed the bge-small cosine as an S6 *rerank* term only reorders a pool the lexical routes already filled — it cannot recover a target that was `never_retrieved`. A dense route at **retrieval** time can: it searches the whole catalog by meaning, so a paraphrased browsing query still pulls the target into the pool.

The cost is one ONNX encode per turn on CPU. Buyers do not need it. So the hypothesis is: **fuse the dense route into the browsing track only** and leave buying BM25-only — recover the browsing `never_retrieved` tail without paying the encode cost, or regressing buying, on the tracks that are already fine.

## What was built

### S5 dense route (`src/retrieval.py`)

`RetrievalConfig` gains `use_dense` (default `False`), `weight_dense` (0.6), `weight_dense_focused` (0.4), `dense_pool` (300). `retrieve()` gains `embed=None`, `qvec=None`. When `use_dense` is set and a usable `EmbeddingIndex` is passed:

```
_rrf(embed.search(qvec or embed.encode_query(full_text), dense_pool), weight_dense, fused)
if state.override_turn is not None:                               # mirrors the lexical `focused` route
    _rrf(embed.search(encode_query(focused_text), dense_pool), weight_dense_focused, fused)
```

fused by the same RRF as the lexical routes (rank fusion, no score calibration). Any exception in the dense path is swallowed — the lexical pool is returned unchanged. `use_dense=False` is byte-identical to the previous pool.

### Per-track wiring (`starter/agent.py`)

`AgentConfig` gains `buying_retrieval / browsing_retrieval (RetrievalConfig | None, None -> reuse self.config.retrieval)`, exactly mirroring the existing `buying_rerank / browsing_rerank`.

`_retrieval_config(track)` selects them. `_respond` passes the per-track config plus the once-per-turn `qvec` to `retrieve()`. The `__init__` embed-load gate widens to fire whenever any retrieval config (shared or per-track) has `use_dense=True`, not only when `rerank.dense_weight > 0`.

### Sweep rows (`tools/sweep.py`)

| row                                 | config                                                                                            |
|-------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>dense_route_browse</code>     | <code>use_router=True, browsing_retrieval=RetrievalConfig(use_dense=True)</code> — THE hypothesis |
| <code>dense_route_all</code>        | <code>use_router=True, retrieval=RetrievalConfig(use_dense=True)</code> — both-tracks control     |
| <code>dense_route_browse_w10</code> | <code>dense_route_browse</code> but <code>weight_dense=1.0</code>                                 |

## Results — `tools/stress_harness.py`

Baseline is router\_on (BM25-only; use\_dense=False -> the dense path never runs, pool byte-identical to before this change).

| run                                                   | config             | score   | $\Delta$       | browsing never_retrieved | browsing MRR |
|-------------------------------------------------------|--------------------|---------|----------------|--------------------------|--------------|
| official (200)                                        | router_on          | 0.91768 | —              | 0/80                     | 0.855        |
|                                                       | dense_route_browse | 0.91501 | <b>-0.0027</b> | 0/80                     | 0.829        |
|                                                       | dense_route_all    | 0.91323 | <b>-0.0045</b> | 1/80                     | 0.812        |
| paraphrase:heavy+browse-gated (200)                   | router_on          | 0.80071 | —              | <b>10/80</b>             | 0.574        |
|                                                       | dense_route_browse | 0.80026 | <b>-0.0004</b> | <b>10/80</b>             | 0.576        |
| paraphrase:heavy+browse-gated, --targets-generic (21) | router_on          | 0.58621 | —              | 4/7                      | 0.143        |
|                                                       | dense_route_browse | 0.58692 | +0.0007        | 4/7                      | 0.143        |

median\_pool\_rank for the browsing scenario is 13 (paraphrased) / 7 (official) with and without the dense route — the fused pool positions are unchanged.

## Reading it

**The hypothesis failed on both counts.** The dense retrieval route:

- recovers **none** of the never\_retrieved browsing targets — 10/80 stays 10/80 under paraphrase, 4/7 stays 4/7 on the generic subset;
- and it is **net negative on the cooperative case** (-0.0027 browse-only, -0.0045 both-tracks; browsing MRR 0.855 -> 0.829): a 4th RRF route full of semantically-plausible belts *dilutes* the lexical ranking of a target that was already sitting at pool rank ~7.

Two things had to both hold for it to work, and the second doesn't:

- The dense route fires.* It does — EmbeddingIndex.available is True, one ONNX encode per turn, RRF-fused at weight\_dense=0.6 (and dense\_route\_browse\_w10 pushes it to 1.0 — same result).
- bge-small ranks the paraphrased target in its top ~300, better than the lexical routes do.* It doesn't. A gated + heavily-paraphrased browsing query is a handful of vague, reworded tokens ("something with a clasp, in a dark tone"); its embedding lands near *hundreds* of belts, and the specific target — whose own text is generic — is not among the 300 closest. Same blind spot as the rerank experiment (docs/team/dense\_rerank.md): the embedding space is dominated by the category, so it cannot separate the target on the paraphrased attributes, whether it is *scoring* a pool (rerank) or *building* one (retrieval).

RRF makes this worse, not better: a route that ranks the target at ~250 contributes  $0.6 / (60 + 250 + 1) \sim 0.0019$  to the fused score — far too little to pull a target the lexical routes put past 300 into the fused top 300.

## What this rules in

- A bigger bi-encoder** (bge-large, e5-large) has the *same* architecture and the same category-dominated space; expect the same result. Not worth GPU time.
- A cross-encoder** (bge-reranker-\*) scores (query, doc) jointly, so it attends to the attribute delta a bi-encoder averages away. But it can only *rerank* a pool something else built — it cannot retrieve — so it is bounded by whatever recall the lexical routes achieve (here, missing 10/80 browsing targets). It is the right next experiment **only in combination with** an upstream recall fix.

- **The upstream fix is not semantic at all:** a paraphrase-robust *query* — synonym / morphology expansion of the customer's tokens before the OR query (leather -> also search hide|cowhide|full-grain), or a category-only retrieval fallback when disclosure is too sparse to form a query. That is what actually addresses never\_retrieved, and it needs no model.

### Combined verdict on bge-small in this repo

| where                    | lever                         | result                                                                         |
|--------------------------|-------------------------------|--------------------------------------------------------------------------------|
| S6 rerank (dense_weight) | cosine as an added score term | +0.02 only on the degenerate tail, -0.003 full set (docs/team/dense_rerank.md) |
| S5 retrieval (use_dense) | cosine as an RRF route        | ~0 on the recall tail, -0.003 to -0.005 on the cooperative case (this doc)     |

The bi-encoder is the wrong tool here in **both** stages, for one reason: the space it works in is dominated by the product *category*, so it cannot separate the target on the paraphrased *attributes* — whether it is scoring a pool or building one. A larger bi-encoder does not change that. The only semantic model that could is a **cross-encoder**, and only as a rerank stage bounded by the lexical routes' recall.

### Status

use\_dense stays False everywhere. The route, the per-track wiring (buying\_retrieval / browsing\_retrieval) and the sweep rows are kept as a measured negative result and as scaffolding for the cross-encoder experiment. The dense\_rerank branch is not proposed for merge.

## Reproduce

```
git checkout dense_rerank
pip install -r requirements.txt
python3 -m unittest discover -s tests -t .
OMP_NUM_THREADS=3 python3 tools/stress_harness.py --customer official --configs router_on,dense_route_browse,dense_route_browse
OMP_NUM_THREADS=3 python3 tools/stress_harness.py --customer paraphrase:heavy+browse-gated --configs router_on,dense_route_browse
OMP_NUM_THREADS=3 python3 tools/stress_harness.py --customer paraphrase:heavy+browse-gated --targets generic --configs router_on,dense_route_browse
OMP_NUM_THREADS=3 python3 tools/stress_harness.py --customer browse-gated --configs router_on,dense_route_browse
```

(dense configs are slow on CPU — one ONNX encode per turn, ~15–25 min per run.)